

PROJECT ARJUNA 2.0 – UNIFIED MASTER TECHNICAL REPORT

Autonomous Real-time Judgment Unit for Navigational Assessment

A Planet-Agnostic Autonomous Landing, Hazard Detection, FDIR, Resource Management & Mission Optimization System

Document Status: Comprehensive master consolidation of 9 source documents (1.txt, 2.txt, 3.txt, 4.txt, 5.txt, 6.txt, 7.docx, 8.pdf, 9.pdf). This document represents the definitive UNION of all project data. No technical information, formulas, specifications, or architectural components have been lost.

1. EXECUTIVE SUMMARY & PROJECT IDENTITY

Project Name: Project ARJUNA (Autonomous Real-time Judgment Unit for Navigational Assessment) **Domain:** Autonomous Systems & Aerospace Engineering Simulation **Project Type:** Software-based autonomous planetary landing simulation, decision-support, fault-recovery, resource-optimization, and ML-training prototype with digital twin integration.

Core Disciplines

- Computer Vision & Machine Learning
- Robotics & Fault-Tolerant Systems
- Physics Simulation & Decision Intelligence
- Generative AI & Mission Planning
- Resource Optimization & Digital Twin Technology

Project ARJUNA is a lightweight, **planet-agnostic** autonomous landing intelligence system designed to simulate how a spacecraft or lander can safely and efficiently navigate and land on different extraterrestrial surfaces.

The central objective is not simply to land safely. ARJUNA attempts to find a solution that maximizes:

Safety + Mission Efficiency + Payload Integrity + Fuel Efficiency + Equipment Preservation + Mission Capability + Resource Utilization

If a soft landing becomes mathematically impossible because of catastrophic failure, ARJUNA transitions from **Landing Mode** → **Survival Mode** and attempts to minimize damage by protecting the most critical payload and equipment.

The fundamental autonomous loop is:

OBSERVE → ESTIMATE → DETECT → ASSESS → OPTIMIZE → ACT → LOG & VERIFY → LEARN

2. PROBLEM STATEMENT

Autonomous landing is one of the most challenging phases of a planetary mission. A spacecraft **cannot assume**:

- The terrain is safe.
- Its sensors will always work.
- The environment behaves like Earth.
- Its original trajectory will remain optimal.
- Wind will remain constant, or engines will provide full thrust.
- Enough fuel will be available for unlimited corrections.

A robust autonomous system must continuously answer:

1. **Environment & State:** What physical environment am I in, and how fast am I moving?
2. **Perception & Reliability:** What terrain and hazards are below me, and can I trust my measurements during extreme environmental conditions?
3. **Landing & Resources:** Where is the safest feasible landing zone, and can I reach it without violating constraints?
4. **Extreme Recovery & Survival:** What should I do if a solar storm causes sensor blackout or an engine fails?
5. **Payload Protection:** If a soft landing becomes impossible, how can I orient the spacecraft to maximize survival of the most critical payload?
6. **Post-Mission Analysis:** How can billions of calculations be converted into a useful human-readable report?

3. CORE CONCEPT & WHY PLANET-AGNOSTIC?

Core Design Philosophy:

The environment is configurable; the intelligence is reusable, and the decisions continuously improve through simulation data.

Different celestial bodies have different physical environments (e.g., gravity changes descent acceleration, atmospheric density changes aerodynamic drag). ARJUNA does not need a separate codebase for every celestial body. It accepts an environment configuration and applies the same core decision architecture.

Example Configurations:

* Mars Config: Gravity: 3.71 m/s²	Atmosphere: Enabled	Base Wind: 12 m/s	Sudden Shear: 45 m/s	Solar Storm Risk: 15%	Terrain: Rocky
* Moon Config: Gravity: 1.62 m/s²	Atmosphere: Disabled	Wind: Negligible	Terrain: Regolith		

4. MULTI-LAYER ARCHITECTURE & SYSTEM FLOW

4.1 Five-Layer Framework

- Layer 1: Environment Simulation:** Planetary Model (6-DOF), Atmospheric Model (3D CFD), Space Weather & Radiation (Solar Wind, CME), Terrain/Surface Model, Gravity Field (Mascon), Illumination (Ray Tracing).
- Layer 2: Physics Simulation:** 6-DOF Dynamics (Quaternions), Orbital Mechanics, Propulsion, Aerodynamics, Thermal Model, Structural Model (Flexible Body Dynamics).
- Layer 3: Sensor Simulation:** Physical Sensor Models, Optical Sensors, Radar/LiDAR, IMU/Gyro/Accelerometer, Star Tracker, Thermal/IR, Colored Noise, Failure & Degradation Models.
- Layer 4: Perception & Intelligence:** Deep Learning (YOLOv9/ResNet-50/ViT/DeepLabV3+), Classical CV Pipeline, SLAM (VI-SLAM/iSAM2), Terrain Relative Navigation, FDIR Engine, Hierarchical RL.
- Layer 5: Control & Guidance:** MPC, Robust Control (H_{∞}), Adaptive Control (MRAC), Path Planning (A*/RRT*/CHOMP).

4.2 System Data Flow

Environment & Mission Config → Physics Engine → Sensor Engine → Perception Engine → State Estimation → FDIR Engine → Mission Resource Engine → Landing Risk Engine → Guidance Engine → Calculation Logger & ML Training Hook → Verification Engine → Mission Control Dashboard & Database → Digital Twin / LLM Reporting.

5. PHYSICS ENGINE & DEVIATION MATHEMATICS

5.1 6-DOF Dynamics & Angle of Deviation

Evaluates states using 6-Degree-of-Freedom dynamics mapped via unit quaternions to prevent gimbal lock, integrated using **Runge-Kutta 4th order (RK4)** with a real-time factor exceeding 10x (GPU-accelerated).

- **Gravitational Force:** $F_g = mg$
- **Thrust Acceleration:** $a_{thrust} = T/m$
- **Net Acceleration:** $a_{net} = T/m - g$
- **Atmospheric Drag:** $F_D = \frac{1}{2}\rho v^2 C_D A$
- **Total Mass Constraint:** $M_{total} = M_{dry} + M_{fuel} + M_{payload}$

A sudden shear wind or asymmetric thrust causes lateral drift. ARJUNA continuously calculates the **Angle of Deviation**:

$$\theta_{dev} = \cos^{-1} \left(\frac{\vec{v}_{desired} \cdot \vec{v}_{actual}}{|\vec{v}_{desired}| |\vec{v}_{actual}|} \right)$$

The system evaluates whether the spacecraft can recover before reaching the surface given available thrust and altitude.

5.2 Flexible Body & Thermal Dynamics

- **Flexible Body:** $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + K(q)q = F_{ext}$. Solar arrays and landing legs cause high-frequency harmonic resonance oscillations requiring filtering.
- **Thermal Model:** $Q_{net} = Q_{solar} + Q_{internal} - Q_{radiated} - Q_{convective}$. Orientation-dependent heat dissipation causes exponential sensor noise increases.

5.3 Propulsion Models

- **Chemical Thrust:** $F_{thrust} = \dot{m} \cdot I_{sp} \cdot g_0$
- **Electric Thrust:** $F_{electric} = \frac{2 \cdot P \cdot \eta}{v_e}$
- **Engine Degradation:** $F_{thrust,actual} = F_{thrust,nominal} \times (1 - degradation_factor)$

- **Fuel Consumption:** $Fuel_{consumed} = \sum (\frac{F_{thrust}}{I_{sp} \cdot g_0}) \cdot dt$

6. SENSOR SIMULATION & EXTREME FAULT INJECTION

6.1 Sensor Models & Colored Noise

Injects **colored noise** mimicking physical COTS and rad-hard components:

- White noise ($\alpha = 0$), Pink 1/f noise ($\alpha = 1$), Brown random walk ($\alpha = 2$), and Markov correlated noise.
- **Camera:** Ray Tracing with Bidirectional Reflectance Distribution Functions (BRDF), PSF blur, and HDR dynamic range simulation.
- **LiDAR:** Simulates rotating scan patterns, point cloud generation, and multi-beam capabilities.
- **IMU:** Full Allan Variance model with scale factor and misalignment errors.
- **Star Tracker:** Simulated blinding from solar flares.
- **Radar:** Simulates Target Radar Cross Section (RCS) and RF attenuation across rocky topography.

6.2 Failure Modes & Solar Storm Simulation

Injects frozen readings, time-delayed data, corruption, EMPs, and hard failures. **Example**

Extreme Sequence (Solar Storm): External Sensors → Camera (BLACKOUT) → LiDAR (BLACKOUT) → FDIR detects common-mode failure → IMU + Dead Reckoning activated → Temporary autonomous navigation continues.

7. COMPUTER VISION & DEEP LEARNING PERCEPTION

7.1 Classical Computer Vision (OpenCV)

Important distinction: Canny Edge Detection is classical computer vision, not AI. The pipeline:

Terrain Image → Grayscale → Noise Reduction (Gaussian) → Contrast Enhancement → Edge Detection (Canny) → Contour/Shape Detection → Geometric Filtering → Hazard Identification → Safe-Zone Generation.

Hazard Map color coding: **GREEN**=Safe, **YELLOW**=Caution, **ORANGE**=Moderate Risk, **RED**=High Risk.

7.2 Deep Learning Perception Engine

Models are optimized via TensorRT (INT8/FP16) for **sub-10ms** latency:

- **YOLOv9**: Real-time hazard bounding boxes (5ms latency).
- **ResNet-50**: Terrain classification (slip/skid prediction).
- **U-Net**: Crater segmentation.
- **ViT (Vision Transformer)**: Temporal feature extraction.
- **DeepLabV3+**: Dense depth maps and surface normal vectors ($\mathbf{n} = \nabla z / |\nabla z|$) for landing strut topological analysis and rollover prevention.

Hazard Categories: Terrain (craters, slopes), Environmental (shear winds, solar storms), System (sensor failures, engine loss).

8. STATE ESTIMATION, SLAM & NAVIGATION

- **SLAM**: Visual-Inertial SLAM (VI-SLAM) with tight coupling, DBoW2 loop closure, and iSAM2 (Incremental Smoothing and Mapping) for real-time marginalization of old states without Jacobian bottlenecks. ORB-SLAM3 integration.
 - **State Estimation**: Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF with $2n + 1$ sigma points; no Jacobian), Particle Filter (adaptive particle count), and Error State Kalman Filter.
 - **Sensor Fusion**: $\mathbf{h}_{estimate} = w_1 \mathbf{h}_1 + w_2 \mathbf{h}_2 + w_3 \mathbf{h}_3$ (Confidence-weighted).
 - **Terrain Relative Navigation (TRN)**: Uses SuperPoint/SuperGlue for geometric verification and PnP pose estimation to override IMU drift.
 - **Learned Navigation**: Inverse Reinforcement Learning (Maximum Entropy IRL) and apprenticeship learning.
-

9. FDIR (FAULT DETECTION, ISOLATION AND RECOVERY)

9.1 Detection & Isolation

- **Detection**: Model-based residuals, ML autoencoders, CUSUM and EWMA for change-point detection (providing "vital seconds of warning"), and Kalman Innovation testing ($\nu_k^T S_k^{-1} \nu_k > \chi_{threshold}^2$).
- **Isolation**: Gradient Boosting and Directed Graphical Models.
- **Target Performance**: >99% isolation rate within a **100-millisecond** threshold.

9.2 Recovery & Health Management

- **Recovery Strategies:** Graceful Degradation Planning, Self-tuning regulators, "Intentional Degraded Mode" prioritizing survival and telemetry.
- **RUL (Remaining Useful Life):** Predicted using LSTM/GRU and Bayesian inference.

9.3 Partial Engine Loss FDIR Sequence

1. Detect thrust asymmetry (Expected $\neq$ Actual) $\rightarrow$ 2. Calculate new thrust capability $\rightarrow$ 3. Determine new deviation angle $\rightarrow$ 4. Recalculate trajectory $\rightarrow$ 5. Use remaining thrust authority $\rightarrow$ 6. Maintain attitude $\rightarrow$ 7. Search for feasible zone $\rightarrow$ 8. Protect fuel reserves.

10. GUIDANCE, CONTROL & MISSION RESOURCES

10.1 Control & Planning

- **MPC (Model Predictive Control):** Prediction Horizon $N_p = 10 - 50$, Control Horizon $N_c = 3 - 10$. Computation in under 2 milliseconds. Uses acados/FORCES Pro.
- **Robust Control:** H_∞ and Sliding Mode Control.
- **Adaptive Control:** MRAC with extremum seeking.
- **Path Planning:** A*, RRT*, CHOMP, and PRM (Probabilistic Roadmap).
- **Safe Corridor:** $\mathcal{C}_{\text{safe}} = \{x \mid d(x, \text{obstacles}) > \text{safety_margin}\}$. Replans within 100 milliseconds.

10.2 Mission Resource Management

Manages Fuel, Payload, Equipment, Thrust, and Power. **Payload Manifest Example:**

- Spectrometer (12kg, Critical, Active)
- Sampling Drill (18kg, High, Standby)
- Scientific Camera (4kg, Critical, Active)
- Data Recorder (2kg, Critical, Active)

10.3 Landing Risk Engine & Fuel-Aware Planning

Risk Score Formula:

$$Score = w_s S + w_f F + w_p P + w_e E - w_r R - w_c C$$

- (Safety, Fuel, Payload, Equipment, Risk, Cost)*. ARJUNA does not blindly choose the mathematically safest terrain; it weighs the fuel expenditure and payload risk required to reach it.
-

11. SURVIVAL MODE & DAMAGE MITIGATION (CRASH VS. LAND PROTOCOL)

If a soft landing becomes mathematically impossible (probability < 1%), continuing to chase it wastes remaining resources. ARJUNA transitions from **LANDING MODE** → **SURVIVAL MODE**.

Controlled-Crash Sequence:

1. **Acknowledge Failure:** Soft Landing Probability → 0%.
 2. **Payload Prioritization:** Identify critical payload vs. non-critical equipment.
 3. **Attitude Reorientation:** Use remaining attitude authority to orient the lander, using lower-priority components as "crumple zones".
 4. **Impact Mitigation:** Evaluate Impact Force ($mass \times V_{impact} / T_{impact}$), terrain, and structural protection.
 5. **Final Objective:** Maximize the probability of preserving the most valuable mission assets.
- (Note: This is explicitly a simulation of damage-mitigation logic, not a guarantee of physical survival in a real-world scenario).*

12. ADVANCED MISSION CAPABILITIES

- **Swarm Coordination:** Shared annotations, laser/RF/UWB mesh networks, PID corrections, packet loss tolerance.
- **Autonomous Rendezvous & Docking (AR&D):** Visual fiducials, 6-DOF relative pose matrices, micro-burn delta-V, time-to-intercept.
- **ISRU (In-Situ Resource Utilization):** GPR (Ground Penetrating Radar) signatures for subsurface ice and resource optimization constraints.
- **3D Reconstruction:** High-fidelity mesh generation with point cloud normals.

13. INFRASTRUCTURE: MICROSERVICES, DATA BUS & HPC

- **Microservices:** gRPC / GraphQL in the API layer. Service discovery via Consul/etcd, Load Balancing via NGINX/HAProxy.
- **Data Bus:** Apache Kafka for high-throughput streaming (e.g., `telemetry.raw` (8 partitions), `guidance.commands` (4 partitions)). Redis cache for ultra-fast state retrieval (TTL: `state:current 1s`, `sensor:latest 100ms`).

- **HPC (High-Performance Computing):** NVIDIA A100 or V100 architectures (6912 CUDA Cores, 432 Tensor Cores, 19.5 TFLOPS FP32). Utilizes MPI and OpenMP for distributed computing via Dask. Near-zero clock drift across nodes.
-

14. DASHBOARD, AI PIPELINE & LLM REPORTING

14.1 Dashboard & UI

- **Mission Control Center:** Built with Streamlit, Dash, React, and Next.js.
- **3D Rendering:** CesiumJS / Three.js featuring dynamic terrain, dust storms, and overlay toggles.
- **Advanced Interfaces:** Holographic/Mixed Reality (HoloLens/Meta Quest) with volumetric inspection, synchronized multi-user shared cursors, and role-based permissions.
- **Voice Commands:** e.g., *"Show hazard map", "Zoom to landing zone"*.

14.2 AI & Machine Learning Pipeline

- **Calculation Logger:** Every tick logged (State → Action → Reward) as training data.
- **Reinforcement Learning:** Hierarchical RL using PPO, SAC, TD3, DQN with experience replay buffers and reward shaping.
- **Meta-Learning (MAML):** $\theta' = \theta - \alpha \nabla_{\theta} L(\theta, task_i)$ for rapid adaptation between Earth-based tests and Martian deployment.
- **Predictive Analytics:** Prophet and ARIMA for time-series, GRU networks, survival analysis.

14.3 LLM Post-Mission Report Generator

Generates a human-readable report summarizing billions of calculations. **Pipeline:** SQLite → Mission Event Extraction (FDIR, Resources, Max Deviation) → Structured JSON → LLM (LangChain, OpenAI, Llama 2, Mistral, Claude) → Post-Mission Report.

- Rule: LLMs summarize logged facts and do not hallucinate events.*

14.4 Digital Twin Integration

Serves as the master node. Real-time "What-If" scenario analysis, astronaut training, and component degradation bridging hardware prototypes with virtual environments.

15. TESTING, VALIDATION & V&V METRICS

- **HIL Testing:** Hardware-in-the-Loop using Raspberry Pi or Jetson boards, automated Pytest fuzzer scripts.
- **Formal Verification:** **nuXmv** (model checking) and **Coq** (theorem proving) ensure critical logic never violates safety constraints.
- **V&V Metrics:** Test Coverage >95%, Requirement Satisfaction 100%, Fault Detection Rate >99%, False Alarm Rate <1%.

Performance Benchmark Results:

- Physics: 1000Hz | Sensor: 500Hz | YOLOv9: 5ms | MPC: 2ms | FDIR: 50ms | REST API: 8ms | WebSocket: <5ms | 3D rendering: 60FPS.
- DB Write Throughput: >50,000 logs/second.
- Scalability (Single to 10-Node): Storage scales 1TB → 100TB, Users 100 → 1000+, Data Rate 100MB/s → 1GB/s.

16. TECHNOLOGY STACK

- **Simulation & Math:** Bullet Physics/MuJoCo, OpenFOAM/SU2, NumPy, SciPy.
- **Perception & SLAM:** PyTorch/ONNX/TensorRT, OpenCV, GTSAM/ORB-SLAM3.
- **Control & Navigation:** acados/FORCES Pro.
- **Data & Backend:** Apache Kafka, Redis, SQLite / PostgreSQL (Hypertables).
- **Frontend:** React/Next.js/Dash, Plotly, Streamlit.
- **AI/LLM:** LangChain, OpenAI, local Llama 2 / Mistral.
- **DevOps & Security:** Docker Compose, Kubernetes, Terraform, Ansible, Helm Charts, GitHub Actions/Jenkins, JWT/OAuth2/Keycloak.

17. COMPLETE DATABASE ARCHITECTURE

The database relies on an optimized schema capable of >50,000 inserts/sec (TimescaleDB/SQLite architecture).

Table Summaries:

1. `missions`: mission_id, target_body, start/end_time, status.
2. `environment`: gravity, atmospheric_density, wind_speed, space_weather_status, terrain_type.
3. `telemetry`: timestamp, altitude, velocity (x,y,z), acceleration, position, fuel, thrust.
4. `sensor_readings`: timestamp, sensor_name, reading, confidence, status, noise_level.
5. `hazards`: timestamp, hazard_type, position (x,y,z), severity, confidence, size, risk_score.

6. `resources`: fuel_remaining, payload_mass, payload_integrity, thrust_available, power_consumed.
 7. `fdir_events`: sensor_or_engine, fault_type, detection/isolation time (ms), recovery_action.
 8. `guidance_events`: current_zone, selected_zone, risk_score, fuel_cost, action.
 9. `calculation_logs`: theta_deviation, thrust_vector (x,y,z), net_acceleration, state_vector, reward.
 10. `mission_reports`: llm_generated_summary, payload_survival_status, mission_outcome, key_events.
 11. `ml_training_data`: state_vector, action_taken, reward, episode.
 12. `digital_twin_state`: physical_state, predicted_state, health_indicators, rul_estimates.
-

18. MISSION WALKTHROUGH & SCENARIO ENGINE

18.1 Example Mars Mission Walkthrough (10 Phases)

- **Initial:** Gravity 3.71 m/s², Atmosphere on, Base Wind 10 m/s, Payload 80 kg, Dry Mass 800 kg.
- **Phase 1-3 (Descent & Assessment):** Camera detects Crater (HIGH RISK) and Boulders. Guidance selects alternative Zone B.
- **Phase 4 (Solar Storm at T+41s):** Camera & LiDAR blackout. FDIR isolates both. IMU + Dead reckoning engaged.
- **Phase 5 (Shear Wind at T+53s):** 45 m/s shear causes lateral drift. θ_{dev} calculated.
- **Phase 6-8 (Engine Anomaly & Replanning):** Partial engine loss reduces thrust. Zone B mathematically infeasible with current fuel/thrust. ARJUNA searches and selects sub-optimal but safe Zone C.
- **Phase 9-10 (Landing & Report):** System lands at Zone C. SQLite logs passed to LLM for Post-Mission Report generation. Outcome: RECOVERED SUCCESS.

18.2 Scenario Engine (10 Predefined Scenarios)

1. Nominal Landing, 2. Crater Detection, 3. LiDAR Failure, 4. IMU Drift, 5. Solar Storm, 6. High Shear Wind, 7. Partial Engine Failure, 8. Fuel Constraint, 9. Payload Protection, 10. Catastrophic Engine Failure (Triggers Survival Mode).

18.3 Mission State Machine

INITIALIZATION → DESCENT → OBSERVATION → HAZARD CHECK → SENSOR HEALTH CHECK → SPACE-WEATHER CHECK → RESOURCE CHECK → ENGINE HEALTH CHECK → LANDING ASSESSMENT

- From Landing Assessment:*

- **SAFE:** CONTINUE → VERIFY → LAND
- **UNSAFE:** REASSESS → CORRECT / NEW ZONE / ABORT
- **CATASTROPHIC:** SURVIVAL MODE / DAMAGE MITIGATION → SURVIVAL OUTCOME

18.4 Mission Success Taxonomy

1. **Normal Success:** Safe landing + payload preserved + sufficient resources.
2. **Recovered Success:** Major anomaly + autonomous recovery + safe landing.
3. **Partial Mission Success:** Landing achieved but non-critical equipment lost.
4. **Survival Success:** Soft landing impossible + controlled damage mitigation + critical payload preserved.
5. **Mission Failure:** Critical payload lost, no viable recovery.

19. PROJECT MANAGEMENT

19.1 7-Day Development Plan

- **Day 1:** Architecture, Repo, Configs, Streamlit skeleton, SQLite schema.
- **Day 2:** Physics engine (gravity, thrust, dynamics, mass) & Resources.
- **Day 3:** Sensors (noise, blackouts), Computer Vision, Terrain preprocessing, Hazards.
- **Day 4:** FDIR (Anomaly detection, fault isolation, recovery logic).
- **Day 5:** Landing Intelligence (Zone risk, fuel-aware planning, Survival Mode).
- **Day 6:** Full Integration (Physics + CV + FDIR + Guidance + DB) & ML/LLM hooks.
- **Day 7:** Testing & Presentation (All 10 scenarios, controlled-crash simulation, report generation).

19.2 Team Distribution

- **Squad 1 (Physics & Perception, 2 members):** Environment models, deviation math, OpenCV, terrain hazard detection.
- **Squad 2 (Simulation & Mission Control, 2 members):** Sensor simulator, Streamlit/Plotly dashboard, telemetry visualization.
- **Squad 3 (Intelligence, Resources & AI, 2 members):** FDIR, sensor fusion, landing-risk engine, guidance, Survival Mode, SQLite logger, ML/LLM integration.

20. PROJECT SCOPE & HONEST TECHNICAL POSITIONING

ARJUNA is explicitly positioned as a **prototype software simulator**, not a flight-certified controller.

- It does not perfectly reproduce planetary physics or guarantee real-world crash survival.
- "Survival Mode" simulates damage-mitigation logic.
- The ML/RL layer operates as an experimental augmentation layer, preserving deterministic safety-critical logic as the primary authority.

Final One-Line Description:

Project ARJUNA is a planet-agnostic autonomous landing intelligence system that continuously analyzes its environment, manages extreme anomalies such as solar storms, sensor blackouts, sudden shear winds and engine loss, performs FDIR, optimizes trajectory and resources for payload preservation, logs its calculations for machine-learning training, and uses Generative AI to produce human-readable post-mission reports.

- (End of Unified Master Technical Report)*

APPENDIX A: COMPLETE DATABASE SCHEMA (EXTENDED)

This appendix preserves the exact structural database schema.

Table: missions

Column	Type	Description
mission_id	SERIAL PRIMARY KEY	Unique mission identifier
target_body	VARCHAR(50)	Target celestial body (Mars/Moon/etc.)
start_time	TIMESTAMP	Mission start time
end_time	TIMESTAMP	Mission end time
status	VARCHAR(20)	Mission status
mission_type	VARCHAR(50)	Type of mission
mission_name	VARCHAR(100)	Mission name
mission_priority	INTEGER	Mission priority level

Table: environment

Column	Type	Description
environment_id	SERIAL PRIMARY KEY	Unique identifier
mission_id	INTEGER REFERENCES missions	Foreign key
gravity	FLOAT	Gravitational acceleration (m/s ²)
atmospheric_density	FLOAT	Atmospheric density (kg/m ³)
atmospheric_pressure	FLOAT	Atmospheric pressure (Pa)
wind_speed	FLOAT	Wind speed (m/s)
wind_direction	FLOAT	Wind direction (degrees)
space_weather_status	VARCHAR(50)	Space weather status
terrain_type	VARCHAR(50)	Type of terrain
solar_radiation	FLOAT	Solar radiation level (W/m ²)
magnetic_field	FLOAT	Magnetic field strength (T)
dust_density	FLOAT	Dust/storm density

Table: telemetry (Hypertable – TimescaleDB)

Column	Type	Description
telemetry_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMPTZ	Time of reading (TIME INDEX)
mission_id	INTEGER REFERENCES missions	Foreign key
altitude	FLOAT	Current altitude (m)

Column	Type	Description
velocity	FLOAT	Current velocity magnitude (m/s)
velocity_x, y, z	FLOAT	Velocity XYZ-components (m/s)
acceleration	FLOAT	Current acceleration (m/s ²)
position_x, y, z	FLOAT	XYZ-coordinates (m)
fuel	FLOAT	Remaining fuel (kg)
thrust	FLOAT	Current thrust (N)
attitude_q0..3	FLOAT	Quaternion components 0-3
angular_velocity_x,y,z	FLOAT	Angular velocity XYZ (rad/s)
mass_total	FLOAT	Total mass (kg)

Table: sensor_readings (Hypertable)

Column	Type	Description
reading_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMPTZ	Time of reading (TIME INDEX)
mission_id	INTEGER REFERENCES missions	Foreign key
sensor_name	VARCHAR(50)	Name of sensor (INDEX)
sensor_type	VARCHAR(50)	Type of sensor
reading	FLOAT	Sensor reading value
reading_raw	FLOAT	Raw reading (before processing)
confidence	FLOAT	Sensor confidence score (0-1)

Column	Type	Description
status	VARCHAR(20)	Sensor status (INDEX)
noise_level	FLOAT	Estimated noise level
calibration_offset	FLOAT	Calibration offset

Table: hazards

Column	Type	Description
hazard_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMP	Time of hazard detection
mission_id	INTEGER REFERENCES missions	Foreign key
hazard_type	VARCHAR(50)	Type of hazard (INDEX)
hazard_subtype	VARCHAR(50)	Hazard subtype
position_x, y, z	FLOAT	XYZ-coordinates of hazard
severity	FLOAT	Hazard severity score (0-1)
confidence	FLOAT	Detection confidence (0-1)
size	FLOAT	Size of hazard (m)
dynamic_hazard	BOOLEAN	Is hazard dynamic?
risk_score	FLOAT	Calculated risk score

Table: resources

Column	Type	Description
resource_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMP	Time of check

Column	Type	Description
mission_id	INTEGER REFERENCES missions	Foreign key
fuel_remaining, initial, consumed, reserve	FLOAT	Fuel tracking metrics (kg)
payload_mass, max	FLOAT	Payload mass metrics (kg)
payload_integrity	FLOAT	Payload integrity score (0-1)
thrust_available, nominal	FLOAT	Thrust metrics (N)
mission_capability	FLOAT	Mission capability score (0-1)
power_available, consumed	FLOAT	Power metrics (W)

Table: fdir_events

Column	Type	Description
event_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMP	Time of FDIR event
mission_id	INTEGER REFERENCES missions	Foreign key
sensor_or_engine	VARCHAR(50)	Affected component
fault_type	VARCHAR(50)	Type of fault
fault_severity	INTEGER	Severity level (1-5)
detection_method	VARCHAR(50)	How fault was detected
detection_time_ms	INTEGER	Time to detect (ms)
detection_details	TEXT	Detection details

Column	Type	Description
isolation_time_ms	INTEGER	Time to isolate (ms)
isolation_details	TEXT	Isolation details
recovery_action	TEXT	Recovery action taken
recovery_success	BOOLEAN	Was recovery successful?
recovery_time_ms	INTEGER	Time to recover (ms)

Table: guidance_events

Column	Type	Description
event_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMP	Time of guidance event
mission_id	INTEGER REFERENCES missions	Foreign key
current_zone, selected_zone	VARCHAR(50)	Landing zones
zone_latitude, longitude	FLOAT	Landing zone coords
risk_score	FLOAT	Calculated risk score
fuel_cost	FLOAT	Estimated fuel cost (kg)
time_to_landing	FLOAT	Estimated time to landing (s)
action	VARCHAR(50)	Action taken
action_reason	TEXT	Reason for action

Table: calculation_logs

Column	Type	Description
log_id	SERIAL PRIMARY KEY	Unique identifier
timestamp	TIMESTAMP	Time of calculation
mission_id	INTEGER REFERENCES missions	Foreign key
theta_deviation	FLOAT	Angular deviation (degrees)
thrust_vector_x, y, z	FLOAT	Thrust vector XYZ (N)
net_acceleration	FLOAT	Net acceleration (m/s ²)
risk_score_matrix	JSONB	Risk score matrix
state_vector	JSONB	Full state vector
action	VARCHAR(50)	Action taken
reward	FLOAT	Reward value
step_number	INTEGER	Simulation step number

Table: mission_reports

Column	Type	Description
report_id	SERIAL PRIMARY KEY	Unique identifier
mission_id	INTEGER REFERENCES missions	Foreign key
llm_generated_summary	TEXT	LLM-generated mission summary
payload_survival_status	VARCHAR(50)	Payload survival status
mission_outcome	VARCHAR(50)	Final mission outcome (INDEX)

Column	Type	Description
key_events, anomalies_detected, resource_usage	JSONB	JSON blocks of events/anomalies/resources
ml_training_flag	BOOLEAN	Used for ML training?
report_generated_at	TIMESTAMP	Report generation time

Table: ml_training_data

Column	Type	Description
training_id	SERIAL PRIMARY KEY	Unique identifier
mission_id	INTEGER REFERENCES missions	Foreign key
state_vector	JSONB	State vector at decision point
action_taken	VARCHAR(50)	Action taken
reward	FLOAT	Reward received
episode	INTEGER	Training episode number
timestamp	TIMESTAMP	Time of training data capture
scenario_type	VARCHAR(50)	Type of scenario
success_flag	BOOLEAN	Was outcome successful?

Table: digital_twin_state

Column	Type	Description
twin_id	SERIAL PRIMARY KEY	Unique identifier
mission_id	INTEGER REFERENCES missions	Foreign key

Column	Type	Description
timestamp	TIMESTAMP	Synchronization timestamp
physical_state, predicted_state	JSONB	Real vs predicted state vectors
health_indicators, rul_estimates	JSONB	Component health/RUL JSON
deviation_from_nominal	FLOAT	State deviation metric

APPENDIX B: PROJECT DIRECTORY STRUCTURE (EXTENDED)

Microservices Version Directory Layout

```
` ARJUNA_2.0/  
  
  • -- README.md  
  • -- docker-compose.yml  
  • -- kubernetes/ (deployment/, ingress/, configmaps/, secrets/)  
  • -- core/ | +-- physics/ (dynamics_6dof.py, quaternion.py, propulsion.py, aerodynamics.py,  
    thermal.py, structural.py) | +-- sensors/ (camera.py, lidar.py, imu.py, star_tracker.py,  
    radar.py, noise_models.py, sensor_fusion.py) | +-- perception/ (deep_learning/,  
    classical_cv/, slam/, terrain_analysis/) | +-- state_estimation/ (ekf.py, ukf.py,  
    particle_filter.py, factor_graph.py) | +-- control/ (mpc.py, robust.py, adaptive.py,  
    sliding_mode.py) | +-- fdir/ (detection/, isolation/, recovery/) | +--  
    resource_management/ (fuel, payload, equipment, thrust, optimization) | +--  
    mission_planning/ (landing_zone_assessment.py, risk_engine.py,  
    trajectory_optimization.py)  
  • -- services/ | +-- physics_service/, sensor_service/, perception_service/, fdir_service/,  
    control_service/, guidance_service/, reporting_service/  
  • -- api/ (main.py, routes/, websockets/, models/, database/)  
  • -- frontend/ (dashboard_dash/, dashboard_react/)  
  • -- ai_ml/ (rl_training/, llm_integration/, predictive_analytics/, meta_learning/)  
  • -- data/ (terrain/, scenarios/, models/, logs/)  
  • -- tests/ (unit/, integration/, performance/, scenarios/)  
  • -- docs/ (api/, architecture/, user_guide/, developer_guide/)`
```